

Git e GitHub: Do Básico ao Pull Request

Parte 1 – Introdução ao Git e Controle de Versões

INFOESTE

Agenda.

Introdução ao Git

- Controle de versões
- O que é o Git
- Configuração e comandos
- Trabalhando com repositórios locais



Controle de Versões.

Controle de Versões.

O que é.

É um sistema que registra as mudanças feitas em arquivos ao longo do tempo, permitindo o acompanhamento e gerenciamento de diferentes versões.

Como funciona.

Cada alteração no código é registrada como um 'snapshot' no tempo. Cada 'snapshot' é um **commit**, um ponto seguro ao qual você pode retornar.

Ex: Imagine que você está escrevendo um livro. Um sistema de versionamento permite que você salve várias versões: 'Capítulo 1 completo', 'Adicionado novo personagem', 'Revisado parágrafo 3'. Se você não gostar de uma mudança, pode voltar à versão anterior facilmente.





Rational ClearCase

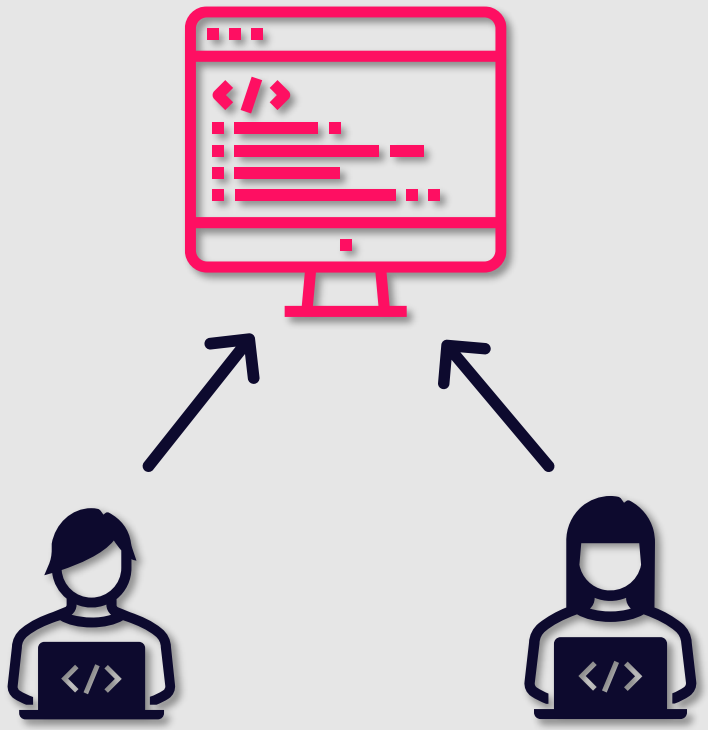




Azure DevOps

Por que usar Controle de Versão?

- Rastreamento de Alterações
- Colaboração Eficiente
- Documentação de Código
- Backup de Código
- Desenvolvimento Paralelo
- Gestão de Versões
- Análise de Erros e Depuração
- Facilidade de Distribuição



Entendendo alguns termos.



Repositório (Repo): É o diretório onde o versionamento acontece.

Staging Area: Área temporária onde você prepara as alterações antes de um commit.

Commit: Uma "foto" salva das suas alterações no histórico do repositório.

Branches (Ramificações): Linhas de desenvolvimento independentes.

Issues (Problemas): Usadas para rastrear tarefas, aprimoramentos ou bugs.

Entendendo alguns termos.



Untracked: Condição em que arquivo ou alteração ainda não foi adicionado a área de preparação.

Flags(-b, -m, ...): São letras/siglas/palavras que atribuem uma variação ou funcionalidade a um comando.

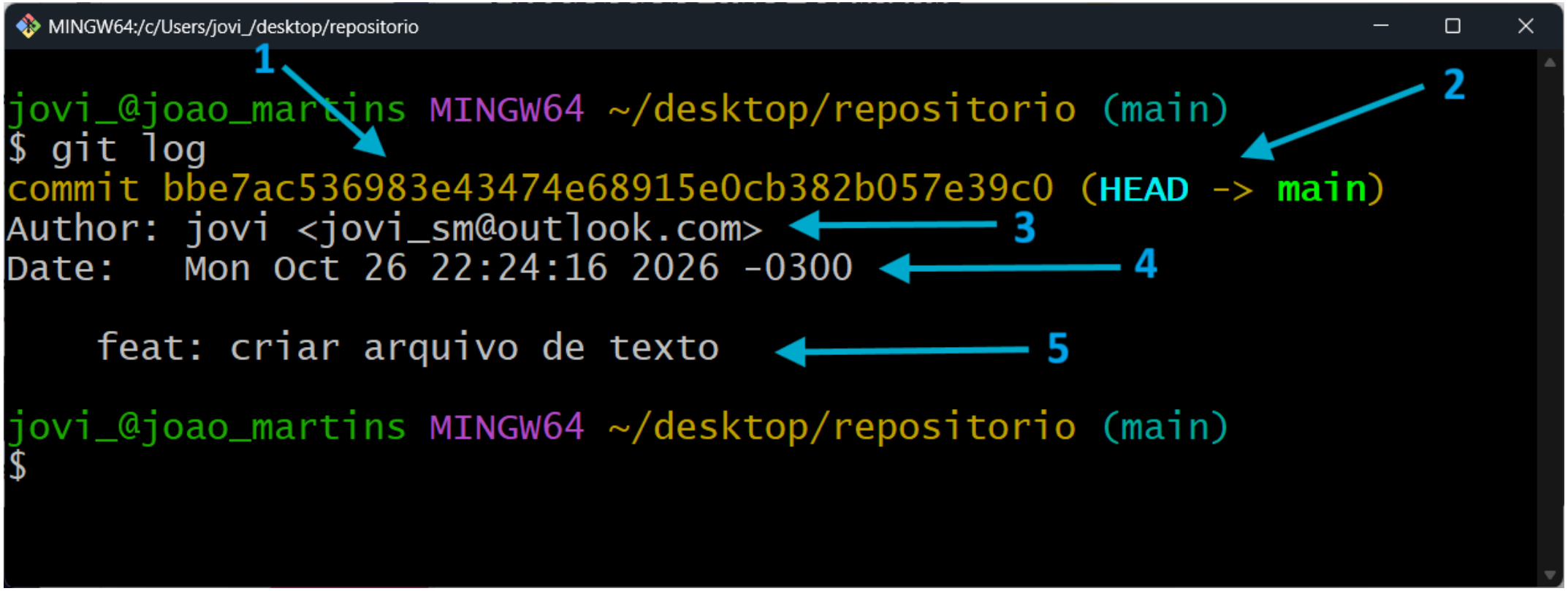
HEAD (Cabeça): Se refere a parte mais recente da *pilha do histórico de alterações.

Pilha: Estrutura em que o último item a chegar é o primeiro a sair.

Master/Main: Nome comumente atribuído a principal branch(ramificação) de um repositório.

Hash do commit: Caracteres que identificam um commit.

Entendendo uma estrutura.




Git.

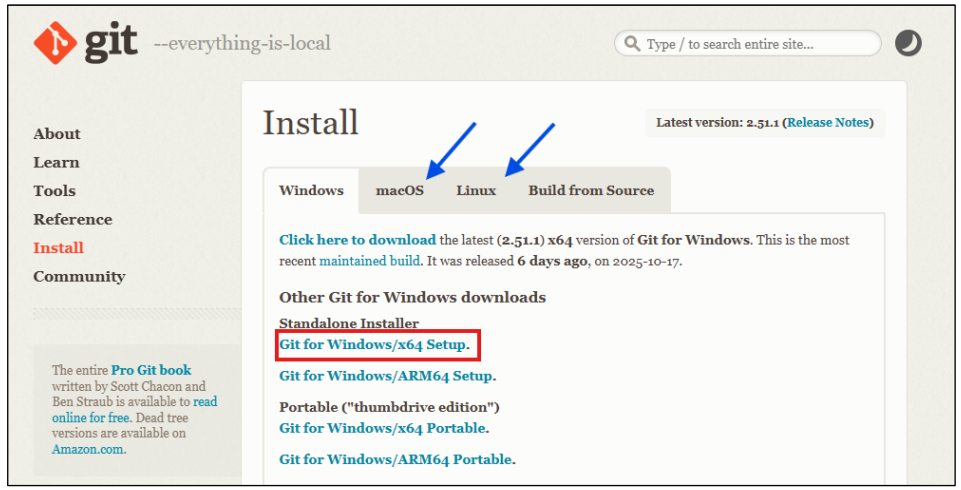
Git.

O que é.

O Git é uma das ferramentas mais importantes no mundo do desenvolvimento. É um sistema de controle de versões distribuído que revolucionou a forma como os desenvolvedores colaboram, acompanham e gerenciam o código-fonte de seus projetos.

Instalação.

Download do Git





Principais comandos - Git.



```
# Definições e configurações
git config

# Iniciar um repositório
git init

# Adicionar arquivos à Staging Area
git add

# Verificar estado do repositório
git status

# Salvar alterações no histórico
git commit

# Guardar alterações temporariamente
git stash

# Visualizar o histórico de alterações
git log

# Ramificações
git branch

# Mudar de ramificação
git checkout

# Unir históricos de ramificações
git merge
```

Comandos úteis – Bash.



```
# Criar um diretório/pasta
mkdir nome_pasta

# Entrar em uma pasta
cd nome_pasta

# Entrar na pasta acima/superior
cd ..

# Criar arquivo
touch nome_arquivo.txt

# Adicionar texto a arquivo
echo “Novo texto...” >> nome_arquivo.txt
```

Configuração.

Após a instalação, devemos realizar algumas configurações iniciais.

Para garantir controle, todo commit é necessariamente identificado com seu autor. Definimos nossas informações de autoria através do comando

`git config`

```
# Define o nome que aparecerá nos commits
git config --global user.name "Seu Nome"

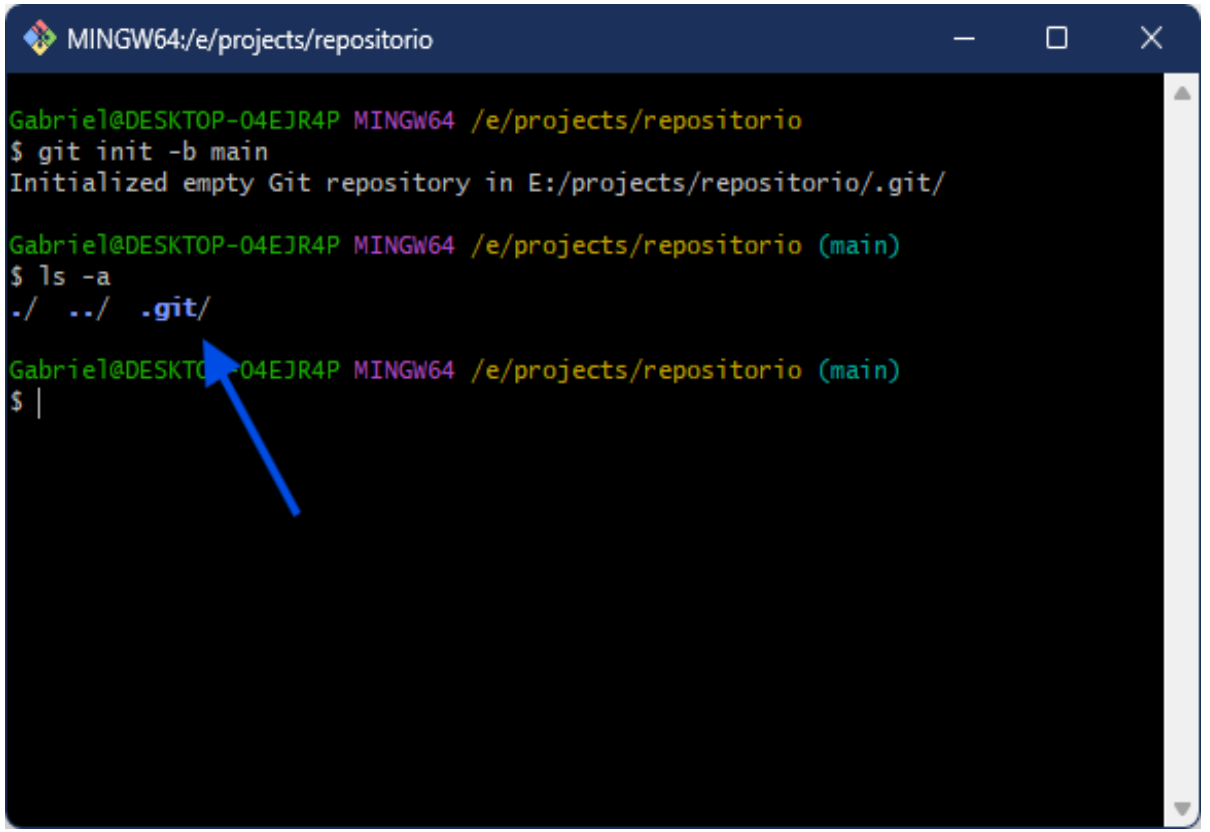
# Define o e-mail que aparecerá nos commits
git config --global user.email "seu-email@example.com"
```

Criando o repositório.

Iniciamos um repositório Git no diretório do projeto onde faremos o controle de versão

```
# Inicia um repositório no diretório em que você está
git init

# Dica de 2026: Inicia o repositório com o nome de branch "main"
git init -b main
```



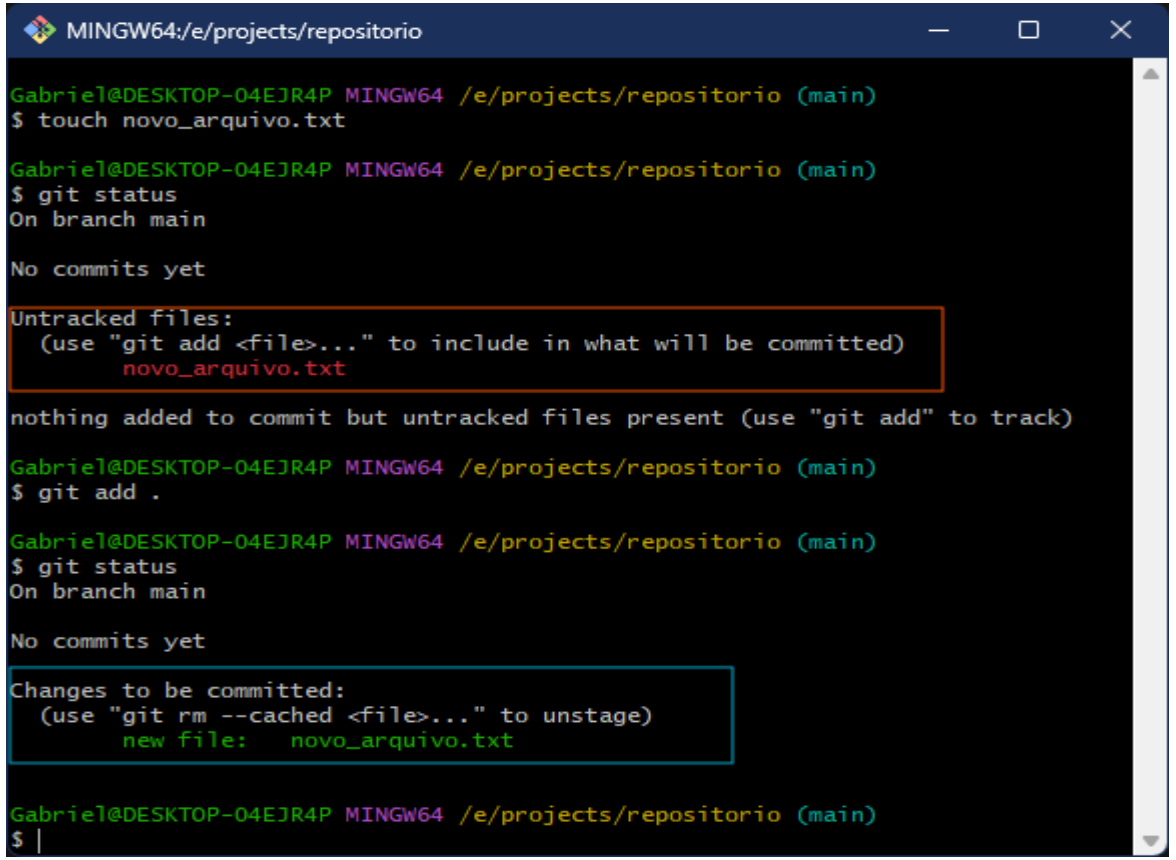
Preparar alterações.

Com o repositório iniciado, todas alterações feitas passam a ser monitoradas pelo Git

Com o comando `git add` adicionamos nossas alterações à “Staging Area”

```
# Adiciona um arquivo específico
git add nome-do-arquivo.txt

# Adiciona todos os arquivos modificados
git add .
```

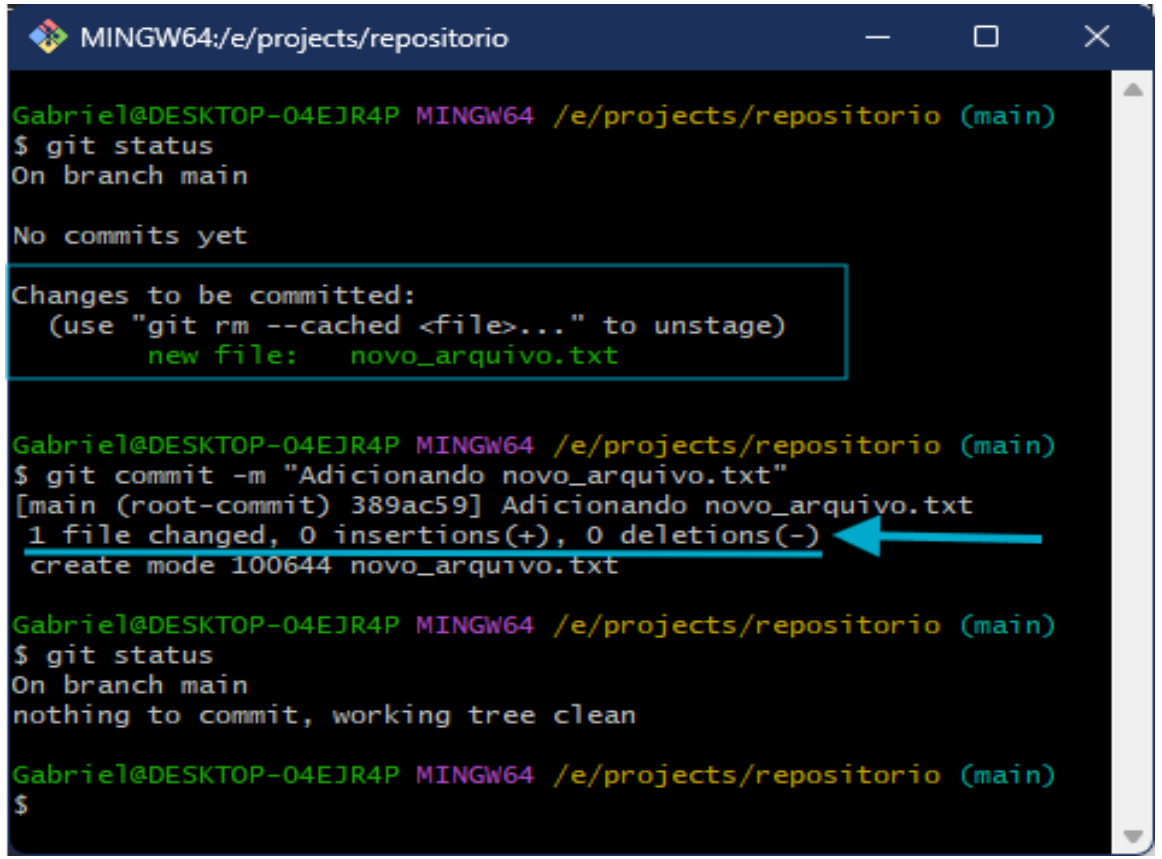


Salvar alterações.

`git commit` pega tudo que está na “Staging Area” e cria um ponto permanente no histórico do projeto.

Cada ***commit*** deve ter uma mensagem que descreve ***o que*** foi mudado. Isso é fundamental para entender o histórico do projeto no futuro.

```
# Salva as alterações da Staging Area no histórico
# O -m significa "message“
git commit -m “feat: uma mensagem que descreve o que foi feito"
```



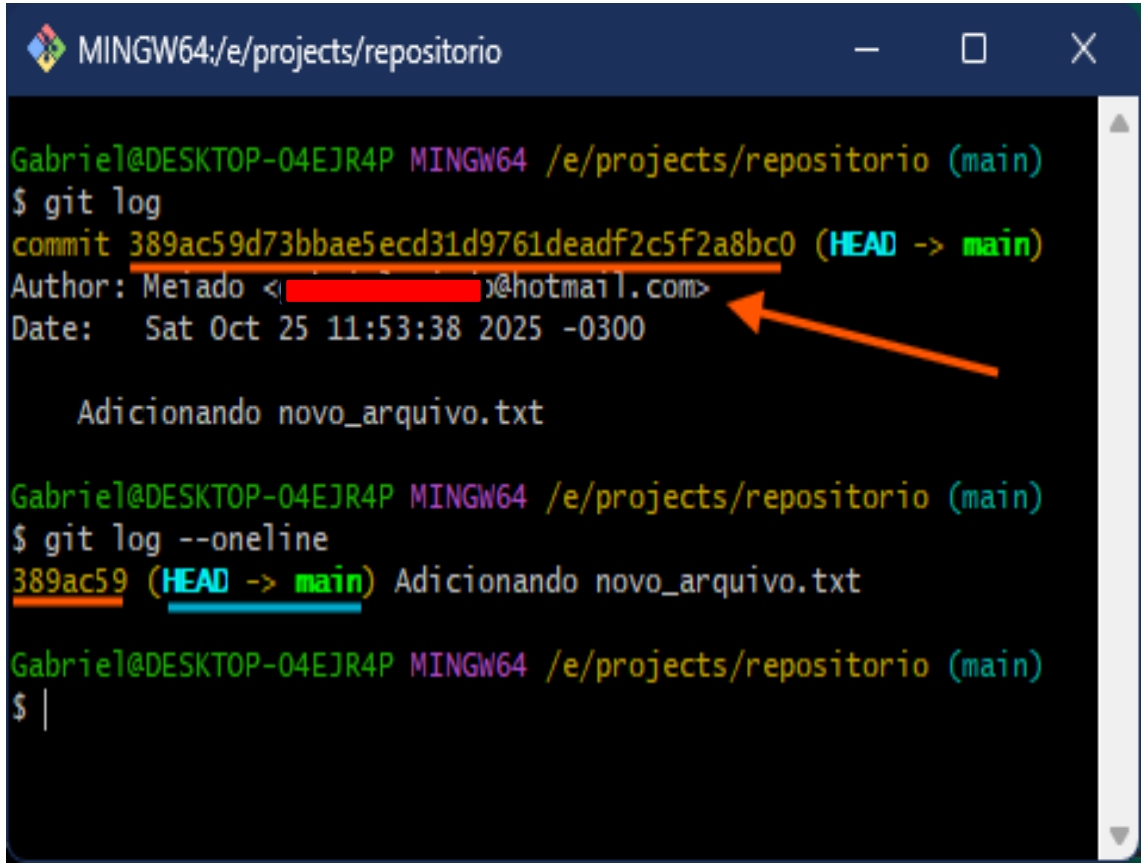
Visualizando Histórico.

O `git log` usado para listar o histórico de commits da ramificação em que estamos, permitindo a visualização de toda evolução do projeto, autores das alterações e quando aconteceram.

```
# Exibe o histórico completo
git log

 # Exibe um log simplificado
git log --oneline

 # Exibe log de últimas 5 alterações
git log -n 5
```

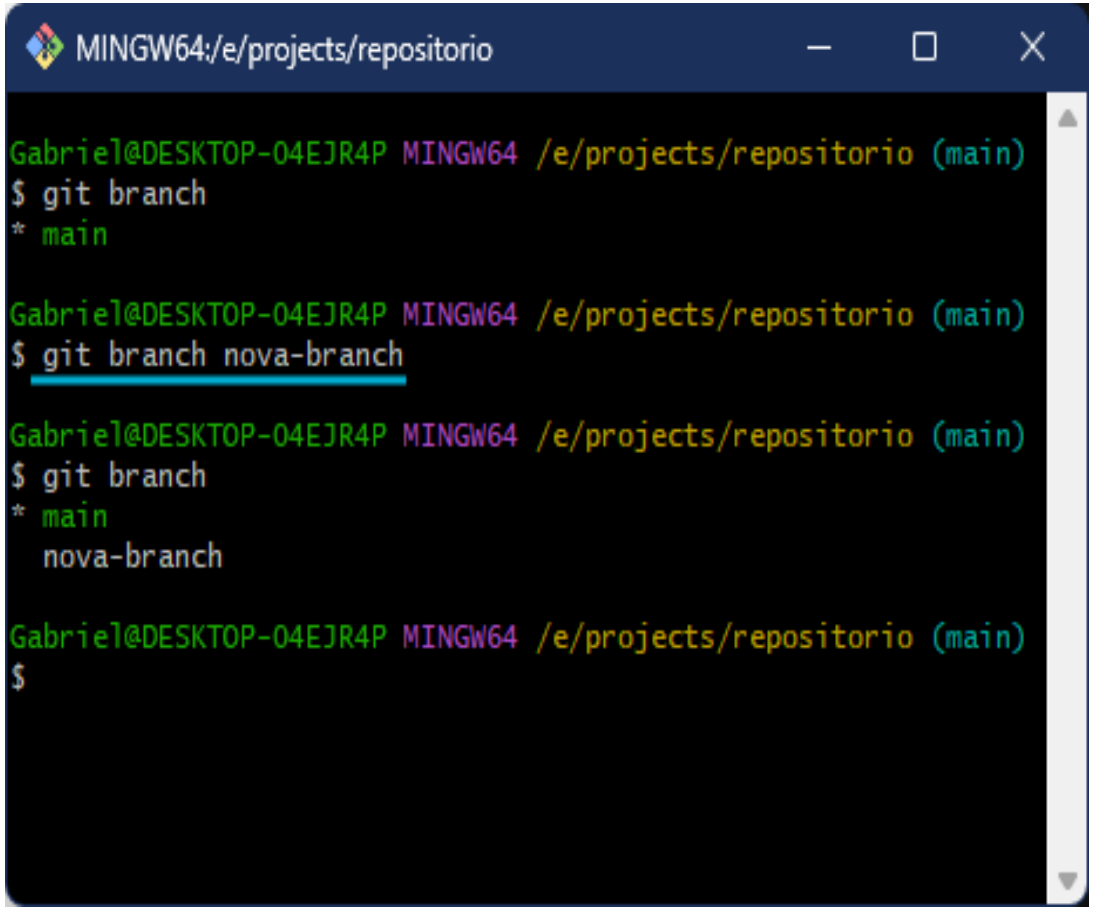


Ramificações.

Ramificações são "linhas do tempo" independentes no seu projeto. Elas permitem que você trabalhe em diferentes funcionalidades ou correções ao mesmo tempo, sem que uma afete a outra.

```
# Lista todas as ramificações locais
# A ramificação marcada com * é a que você está
git branch

# Cria uma nova ramificação com o nome que você informar
git branch nova-branch
```

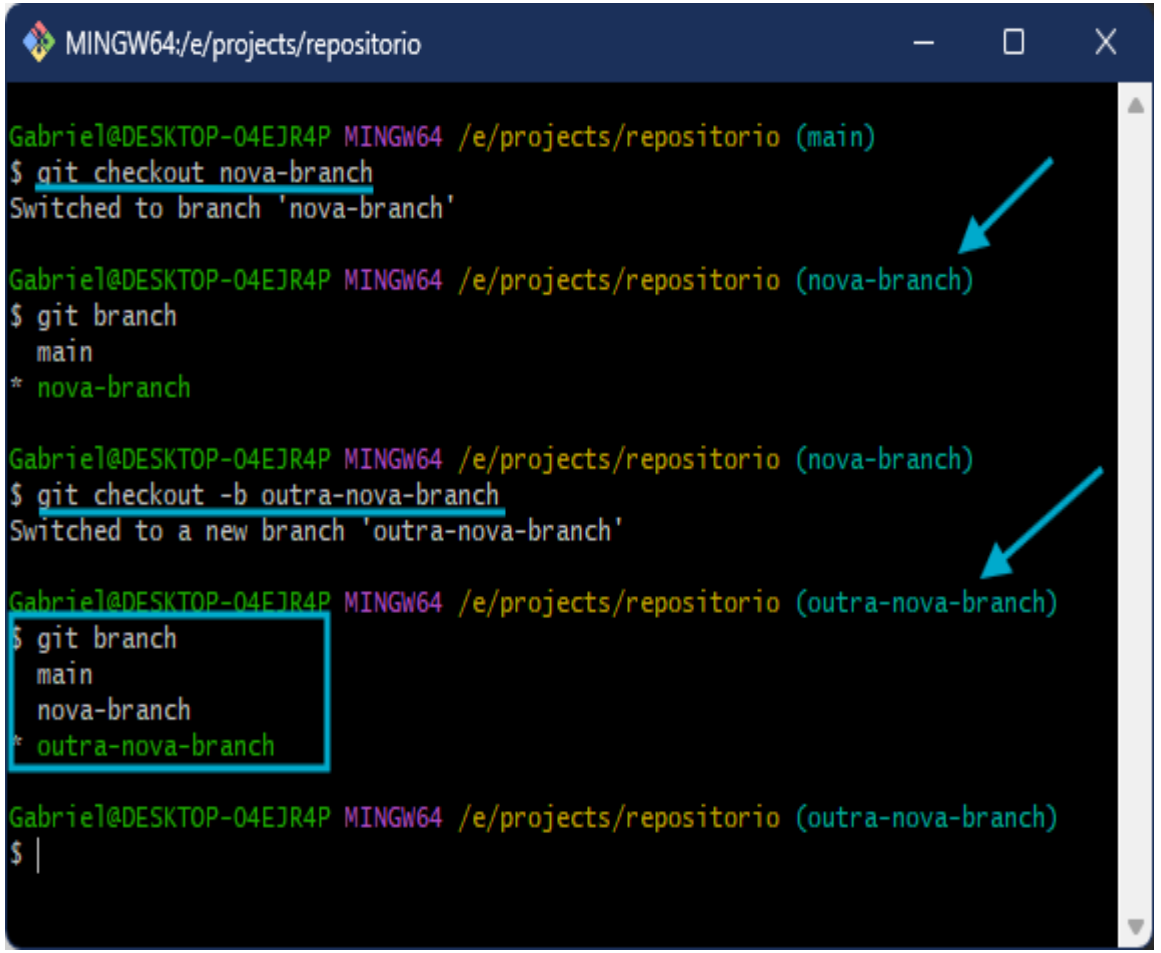


Ramificações.

Agora que temos nossa nova ramificação, precisamos de um jeito de "entrar" nela para começar a trabalhar. O `git checkout` é usado para navegar entre as ramificações (branches) e também entre commits antigos.

```
# Muda da ramificação atual para a "nova-branch“
git checkout nova-branch
#git switch nova-branch

# Atalho para CRIAR e MUDAR ao mesmo tempo
# (Faz 'branch' e'checkout' em um passo)
git checkout -b outra-nova-branch
#git switch –c outra-nova-branch
```



Unir Ramificações.

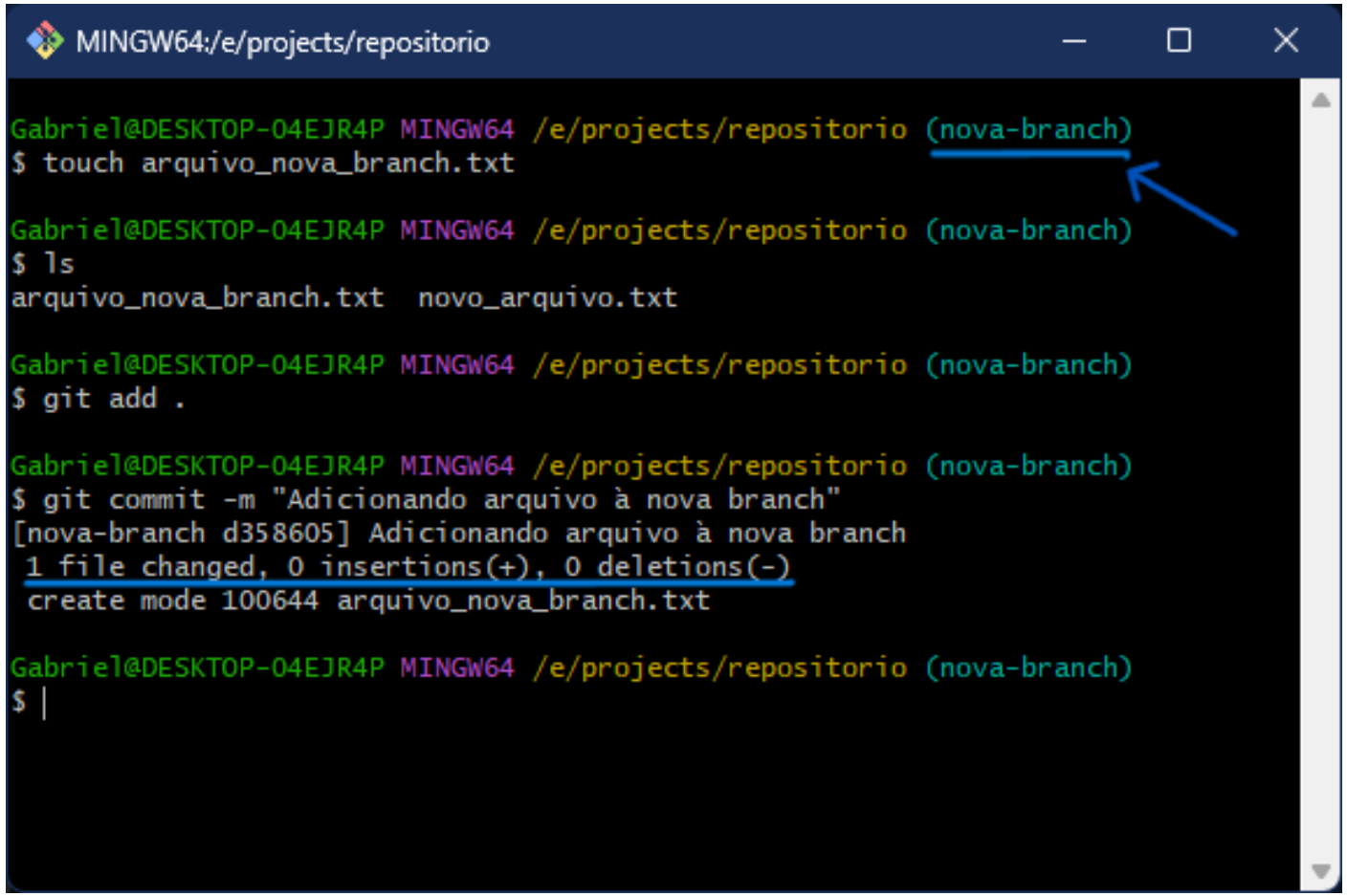
Depois de terminar o trabalho na nova branch, você vai querer trazer as alterações de volta para a branch de onde saiu a ramificação

O `git merge` faz a fusão dos históricos de uma branch na outra

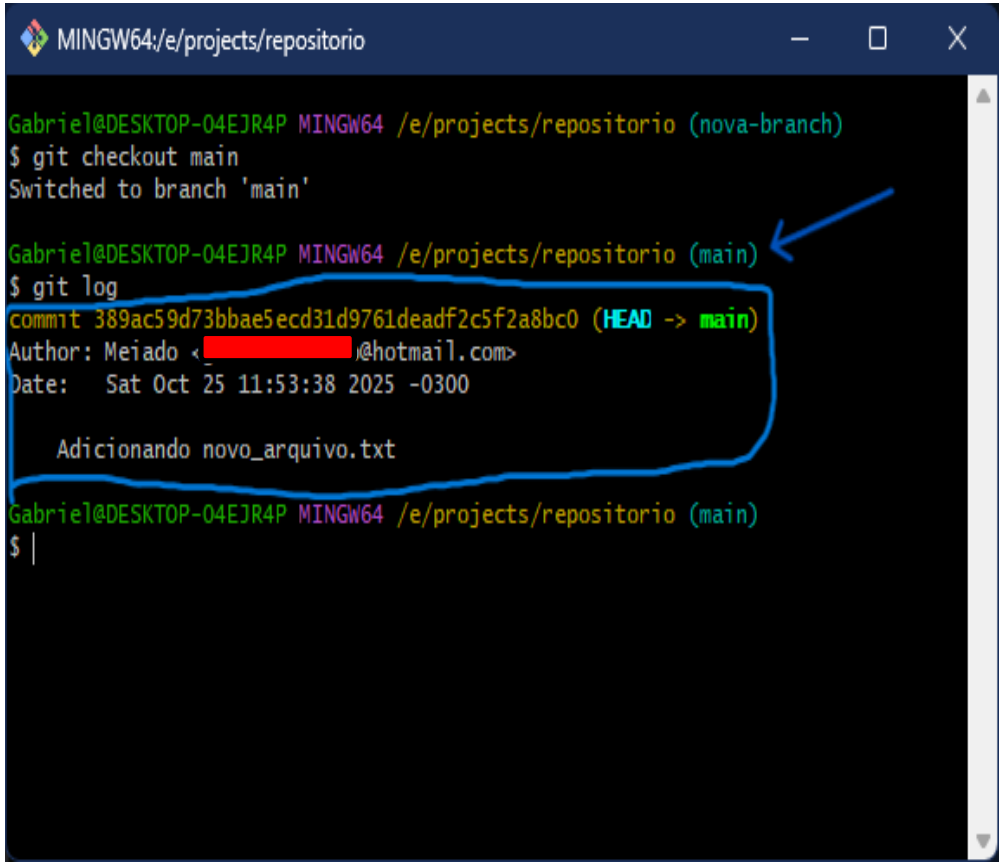
```
# Voltar para a branch original
git checkout main

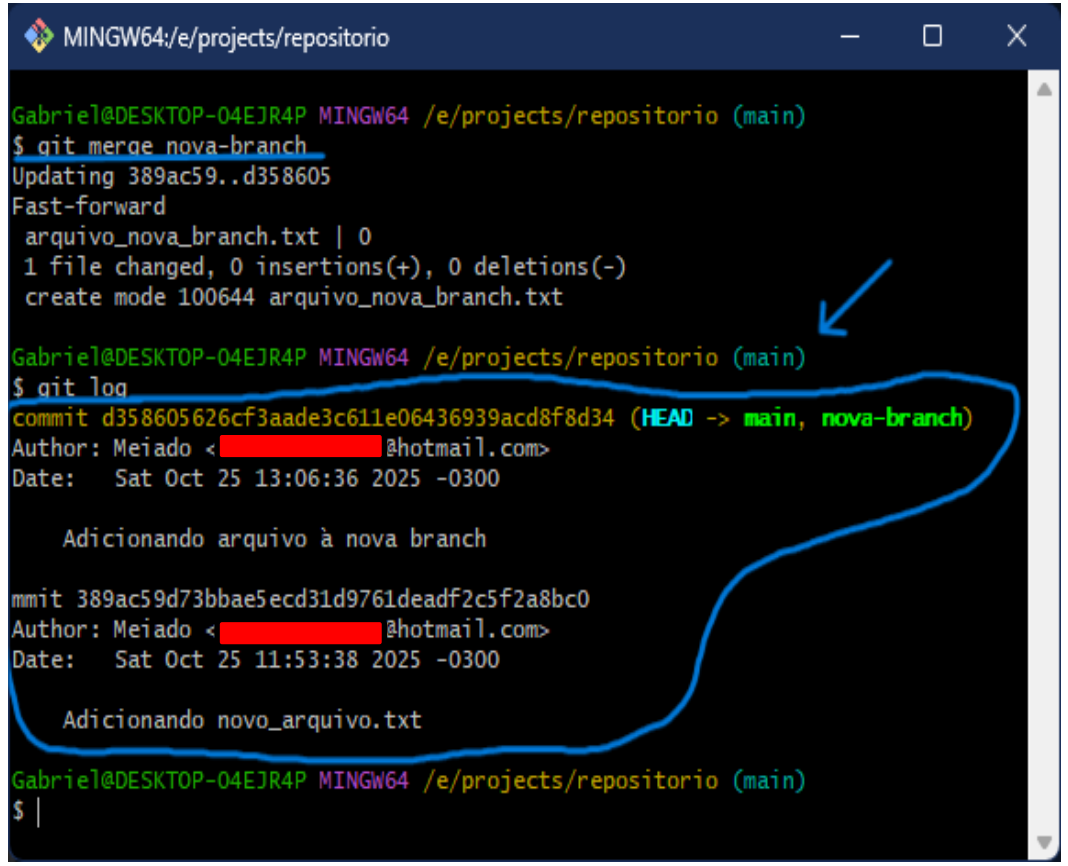
# Trazer as alterações da "nova-feature" para a
"main“
git merge nova-branch
```

Unir Ramificações.



Unir Ramificações.





Reaplicando Ramificações.

Depois de terminar o trabalho na nova branch, você pode querer atualizar a branch principal deixando a alteração alocada na cabeça do histórico.

O `git rebase` faz uma reaplicação do patch de alterações no topo da pilha da branch referenciada.

```
# Atualizar branch principal
git pull origin main

# Voltar para a branch original
git checkout main

# Trazer as alterações da "nova-feature" em cima da "main“
git rebase main
```

Não faça rebase de commits que existem fora do seu repositório e nos quais as pessoas possam ter baseado o trabalho delas.

Se você seguir essa diretriz, ficará tudo bem. Se não o fizer, as pessoas o odiarão, e você será desprezado por amigos e familiares.

Guardar alterações temporariamente.

O `git stash` é o comando que você usa quando está no meio de uma alteração, mas ainda não está pronto para *commitar*. De repente, você precisa trocar de branch, mas o Git não deixa, pois você tem trabalho não salvo.

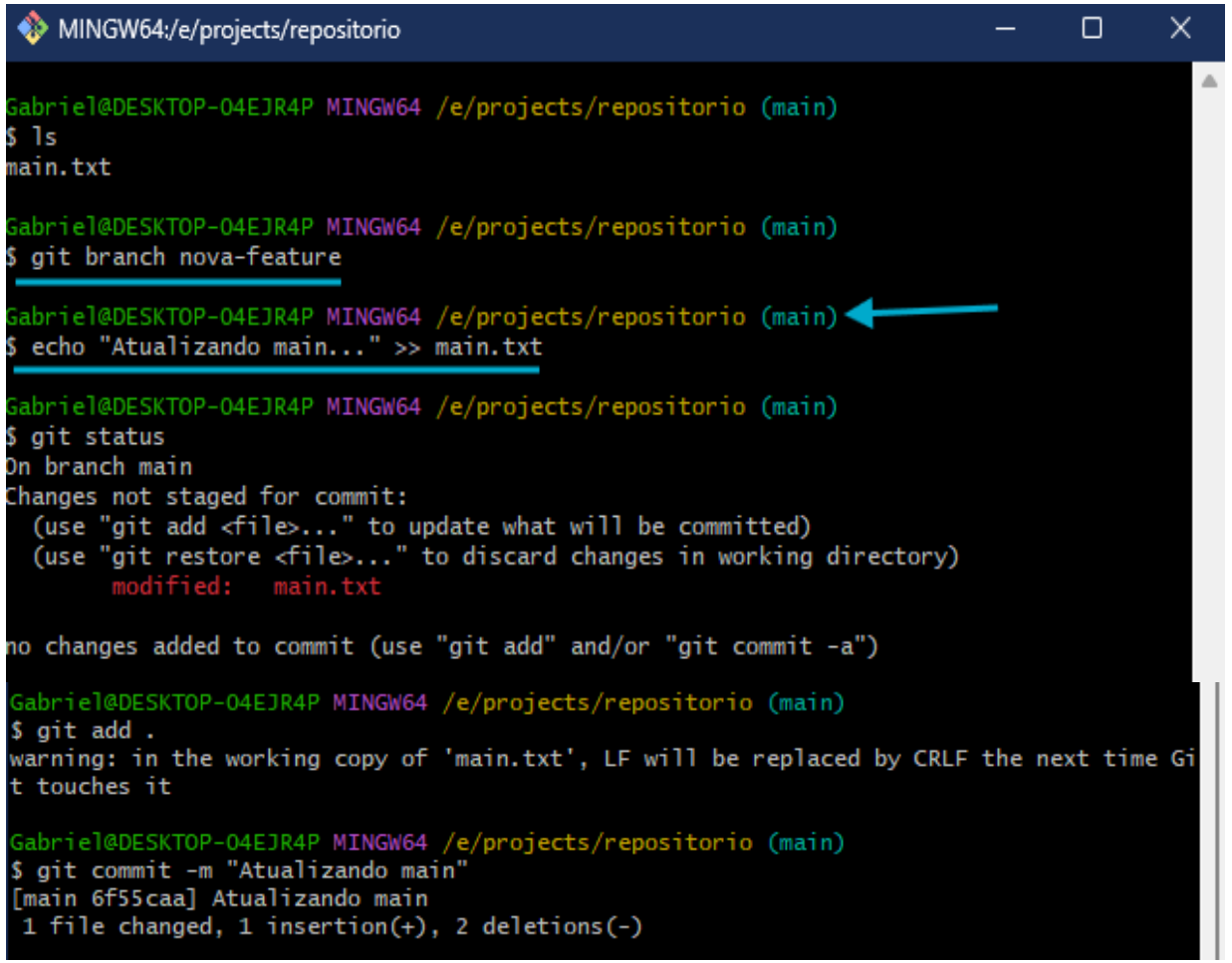
O ***stash*** guarda essas alterações de lado, limpando a branch e permite a troca

```
# Salva as alterações atuais com uma mensagem
git stash push -m "Minha alteração incompleta“

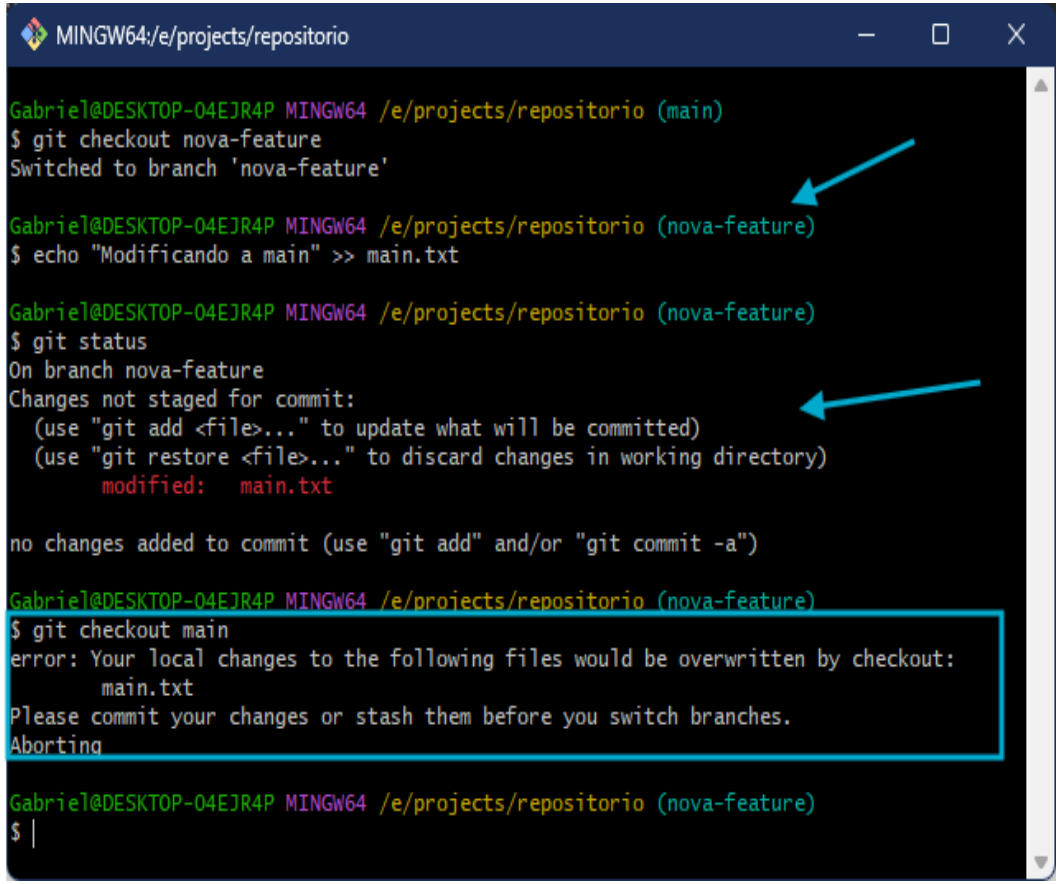
# Lista todos os "stashes" guardados
git stash list

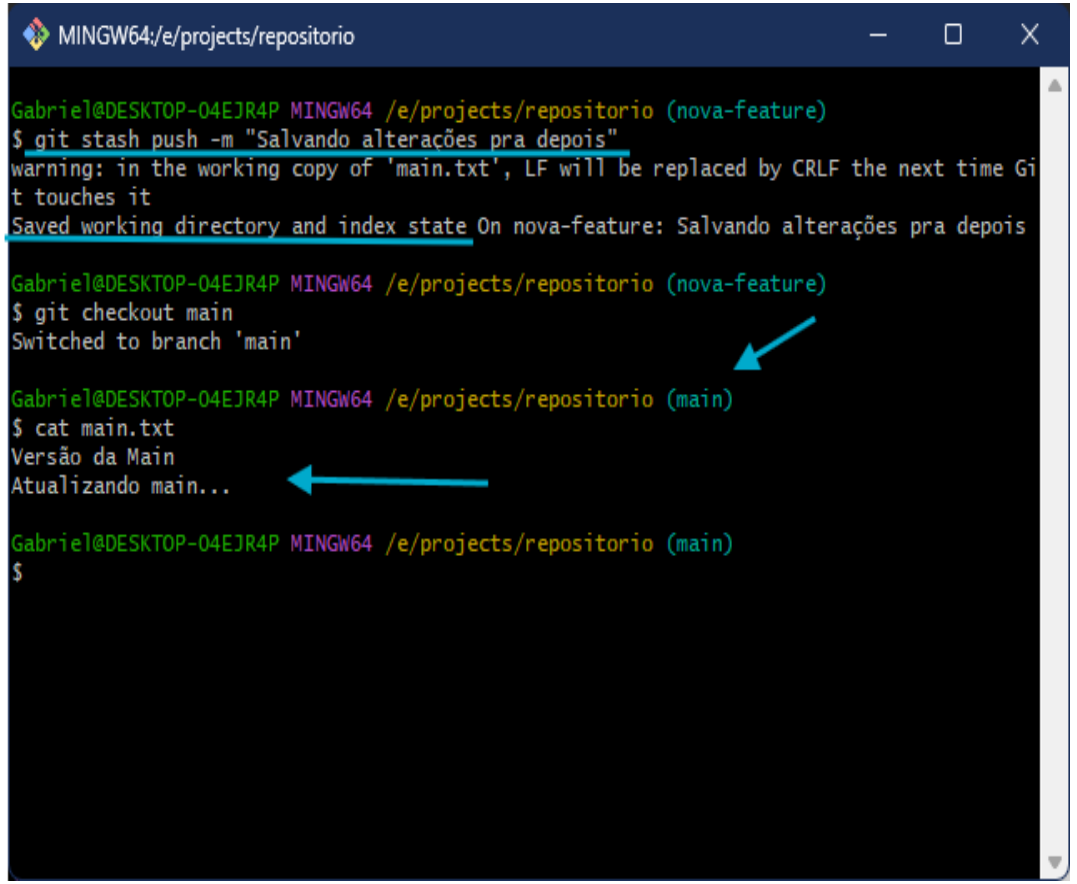
# Aplica o último stash guardado E o remove da lista
git stash pop
```

Guardar alterações temporariamente.



Guardar alterações temporariamente.





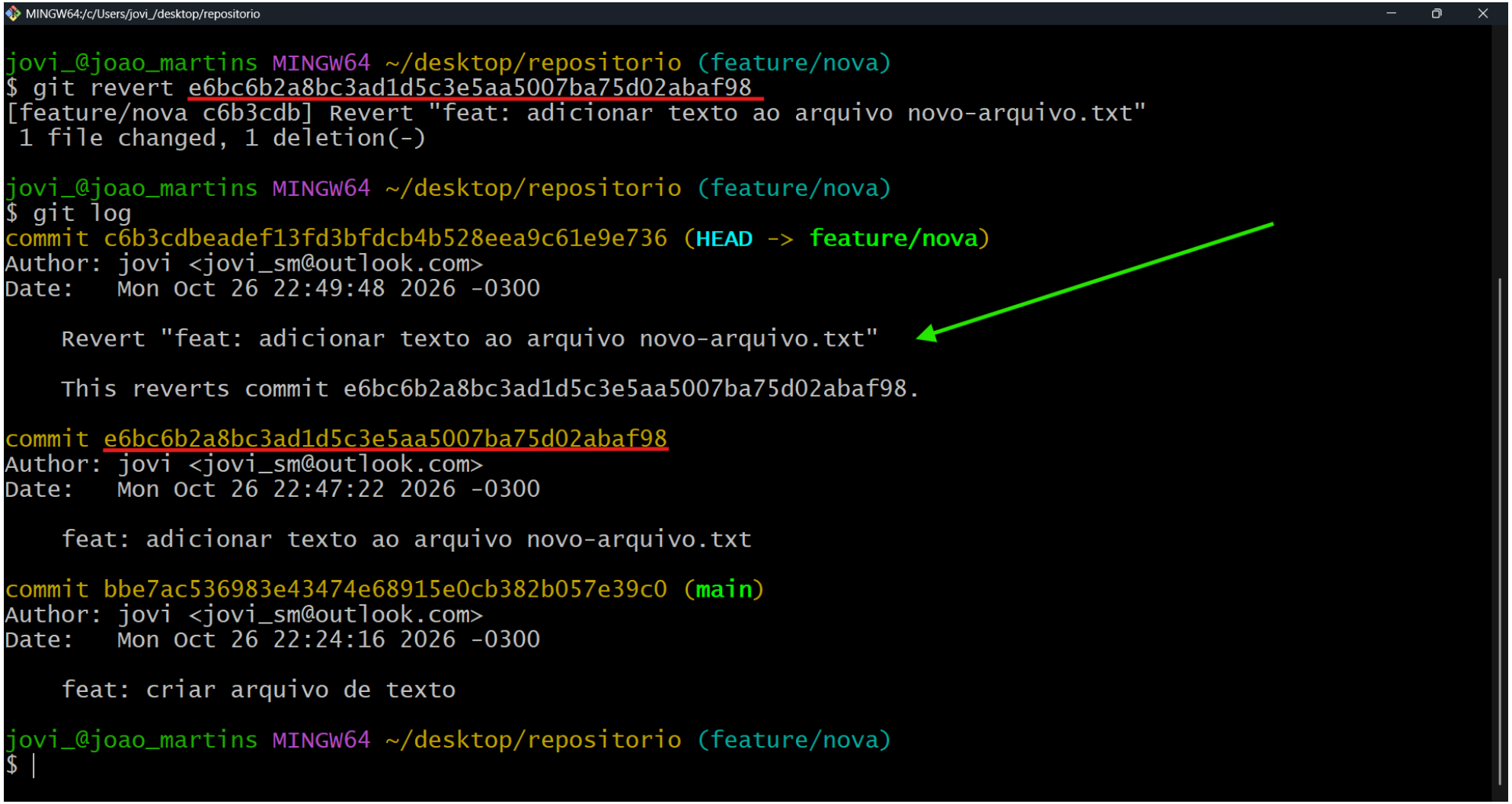
Desfazer alterações.

O `git revert` é o comando que você usa quando um commit já realizado deve ter suas modificações retrocedidas.

O ***revert*** gera um commit que reverte as alterações do commit especificado.

```
# Gera um commit com a reversão das alterações.
git revert <hash do commit>
```

Desfazer alterações.



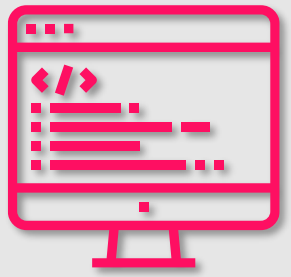
Ignorar arquivos e pastas.

O Git possui um mecanismo para ignorar certos arquivos e pastas no controle de versões, o arquivo `.gitignore`

Essa função é fundamental para evitar que arquivos desnecessários e arquivos sensíveis sejam adicionados ao histórico de alterações:

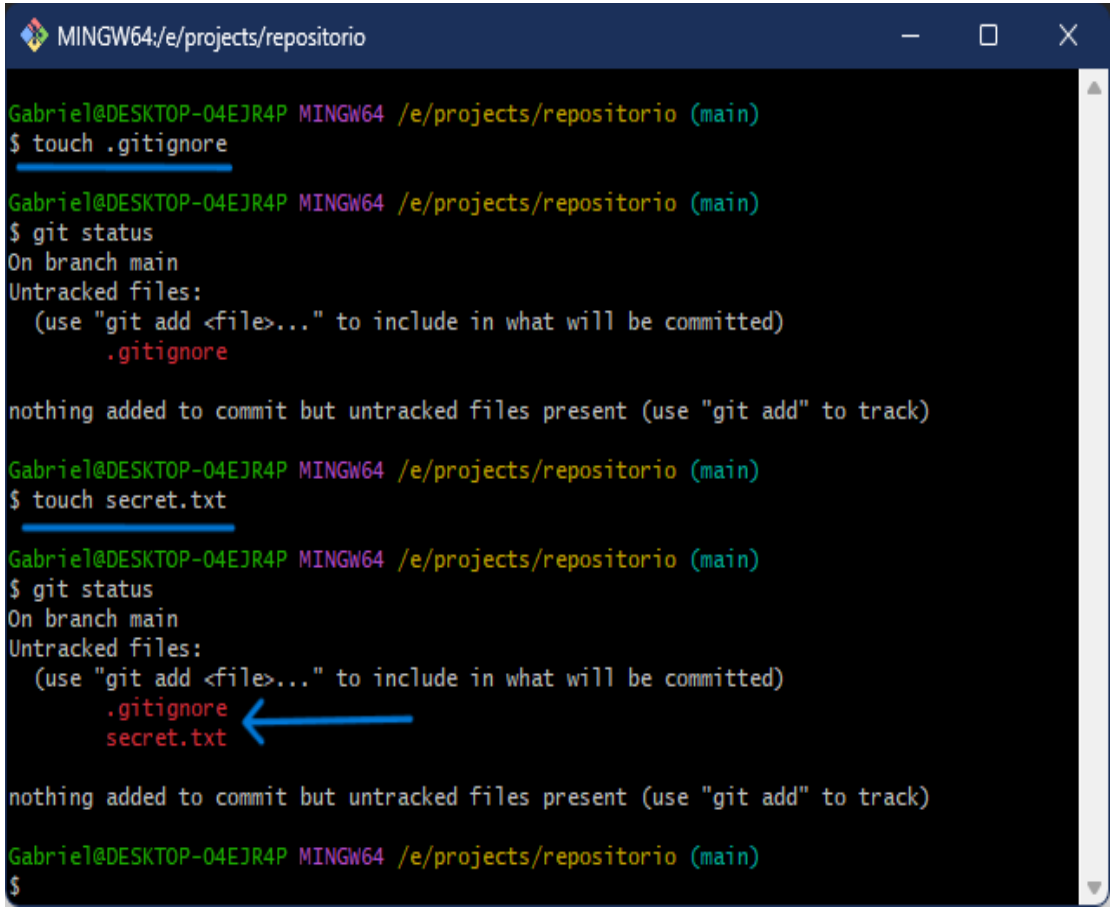
- Pastas de dependências (ex: `node_modules/`)
- Arquivos de segredos (ex: `.env`)
- Arquivos de Sistema (ex: `.DS_Store` no Mac)
- Pastas de Código compilado (ex: `build/` `dist/` `target/`)

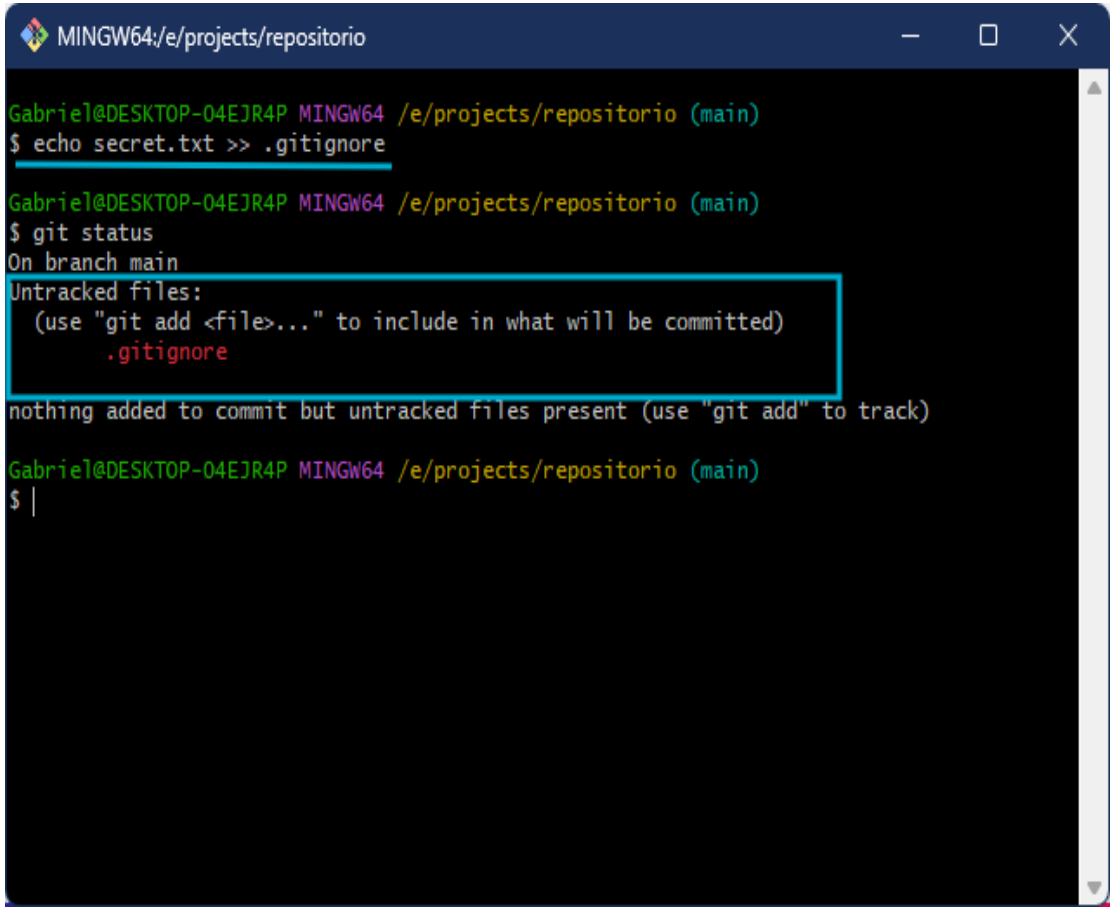




INFOESTE

Ignorar arquivos e pastas.





Dúvidas?

INFOESTE

Obrigado pela presença!

Nos vemos na quinta-feira (29/10), até pessoal!



INFOESTE





Git e Github: Do Básico ao Pull Request

Parte 2 – Colaborando com o Mundo

INFOESTE

Agenda.

Colaborando com o Mundo

- Git x GitHub
- Repositórios remotos
- Trabalhando com projetos existentes
- Fluxo de colaboração
- Colaborando no Mural




GitHub.

GitHub.

O que é.

O GitHub é uma **plataforma** ou **serviço** online. Pense nele como um “serviço de hospedagem na nuvem” feito especificamente para repositórios Git.

Ele fornece um site com uma **interface gráfica** que **facilita a colaboração** entre desenvolvedores, permitindo que equipes trabalhem juntas no mesmo código, acompanhem *Issues* (tarefas/bugs) e façam *Pull Requests*.



Git x GitHub.



- É a ferramenta
- Software de linha de comando
- Instalado no computador
- Controle de histórico de versão

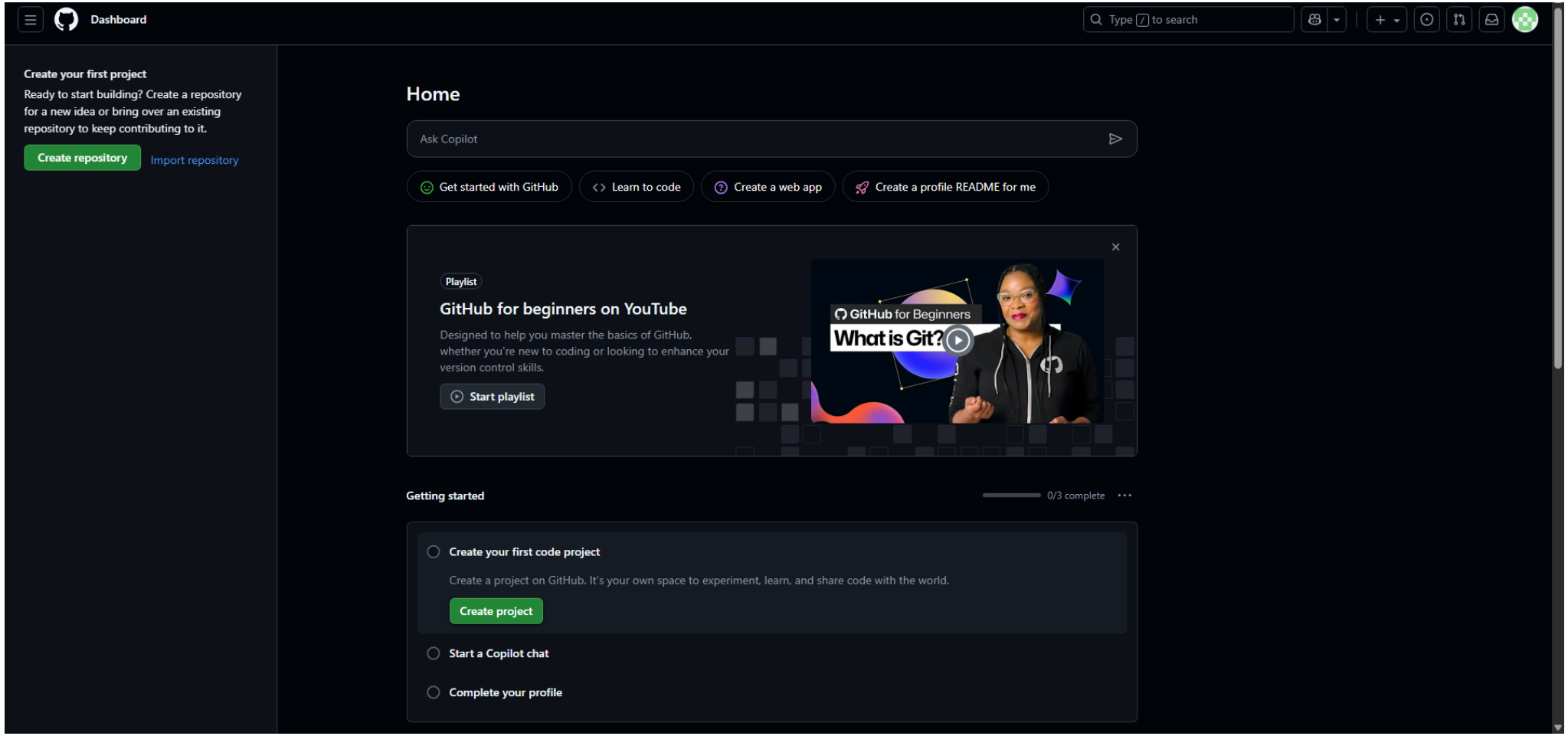


- É o serviço/plataforma
- Site (interface gráfica)
- Hospedado na nuvem
- Facilita a colaboração e hospedagem

INFOESTE

GitHub.

Tour pela plataforma.



Repositórios Remotos.

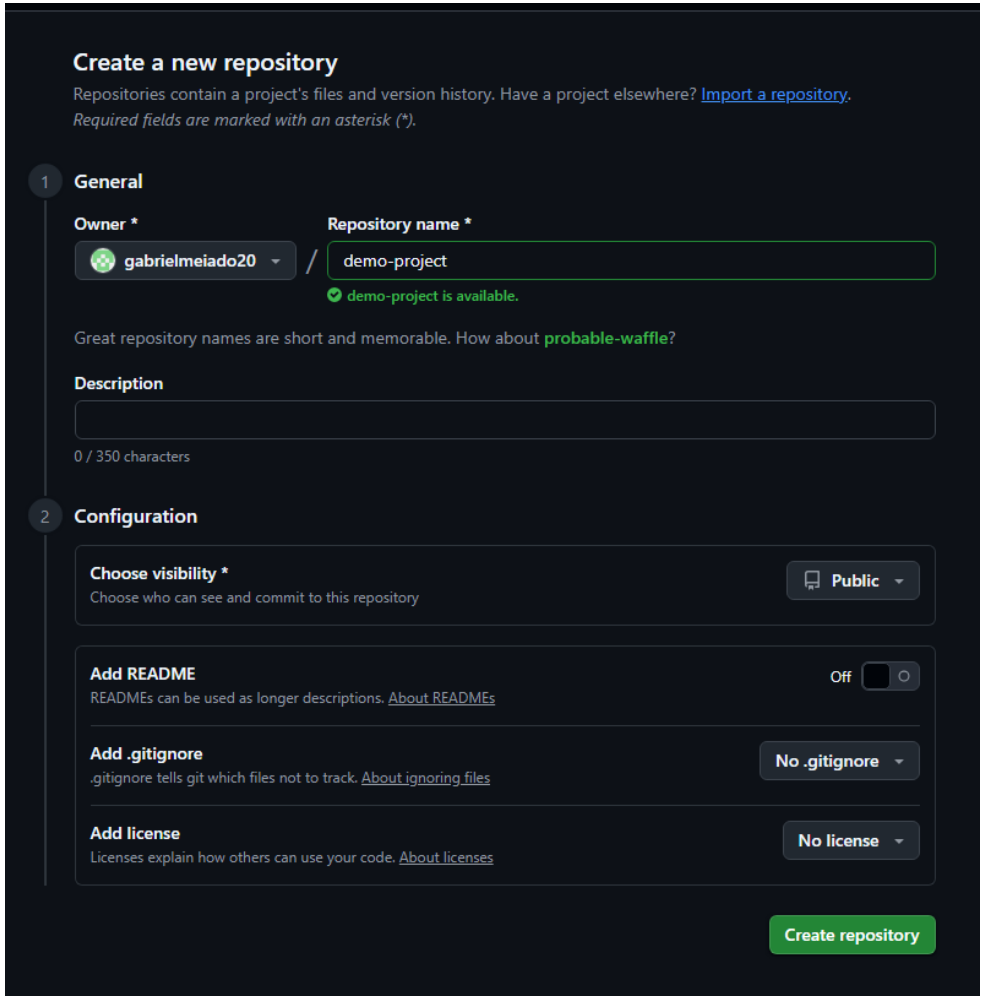
Para que possamos comunicar nossas mudanças locais ao repositório criado na plataforma é necessário sinalizar essa relação e criar a ponte entre os dois

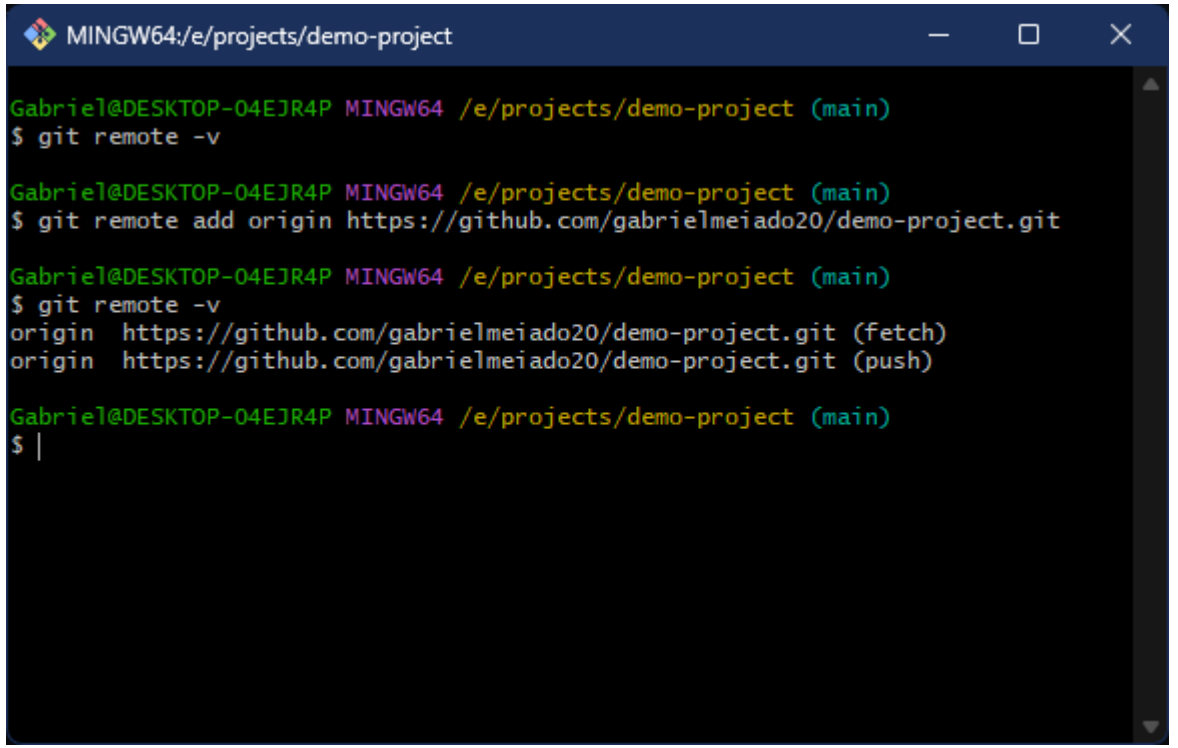
Usamos o comando ***git remote add*** para isso, sinalizando ao Git qual a URL do repositório remoto onde as alterações também serão salvas

```
# Adiciona um "apelido" (remote) chamado "origin“
# para a URL do seu repositório no GitHub
git remote add origin https://github.com/seu-usuario/seu-repo.git

# Para verificar se o "remote" foi adicionado com sucesso
git remote -v
```

Repositórios Remotos.





Repositórios Remotos.

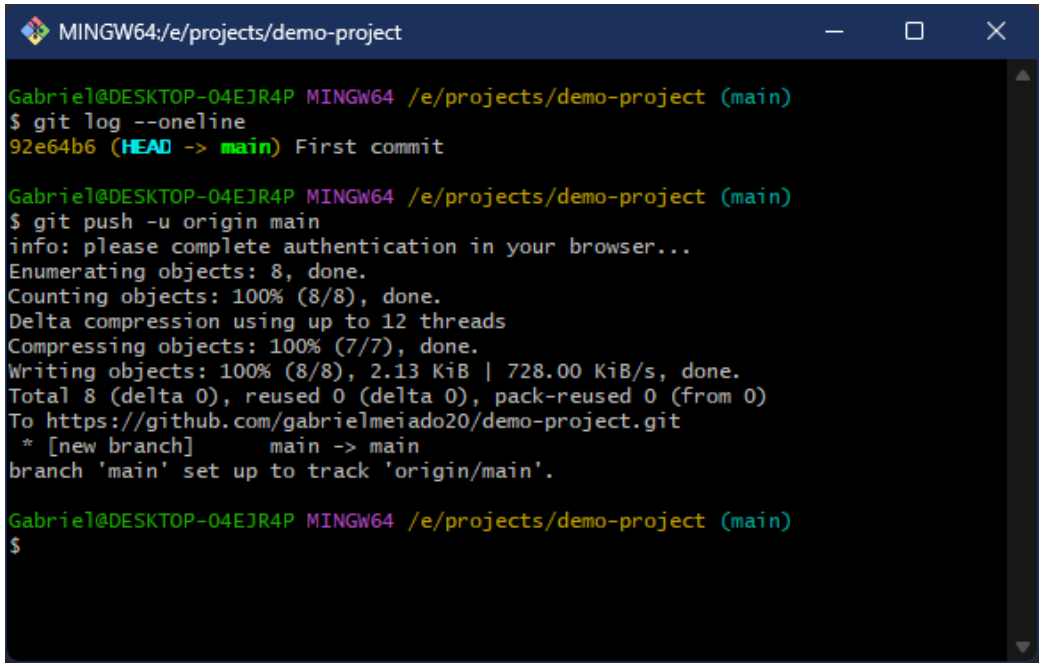
Com o repositório conectado e as alterações já no histórico após os commit, o comando ***git push*** é que finalmente envia as alterações locais para o GitHub

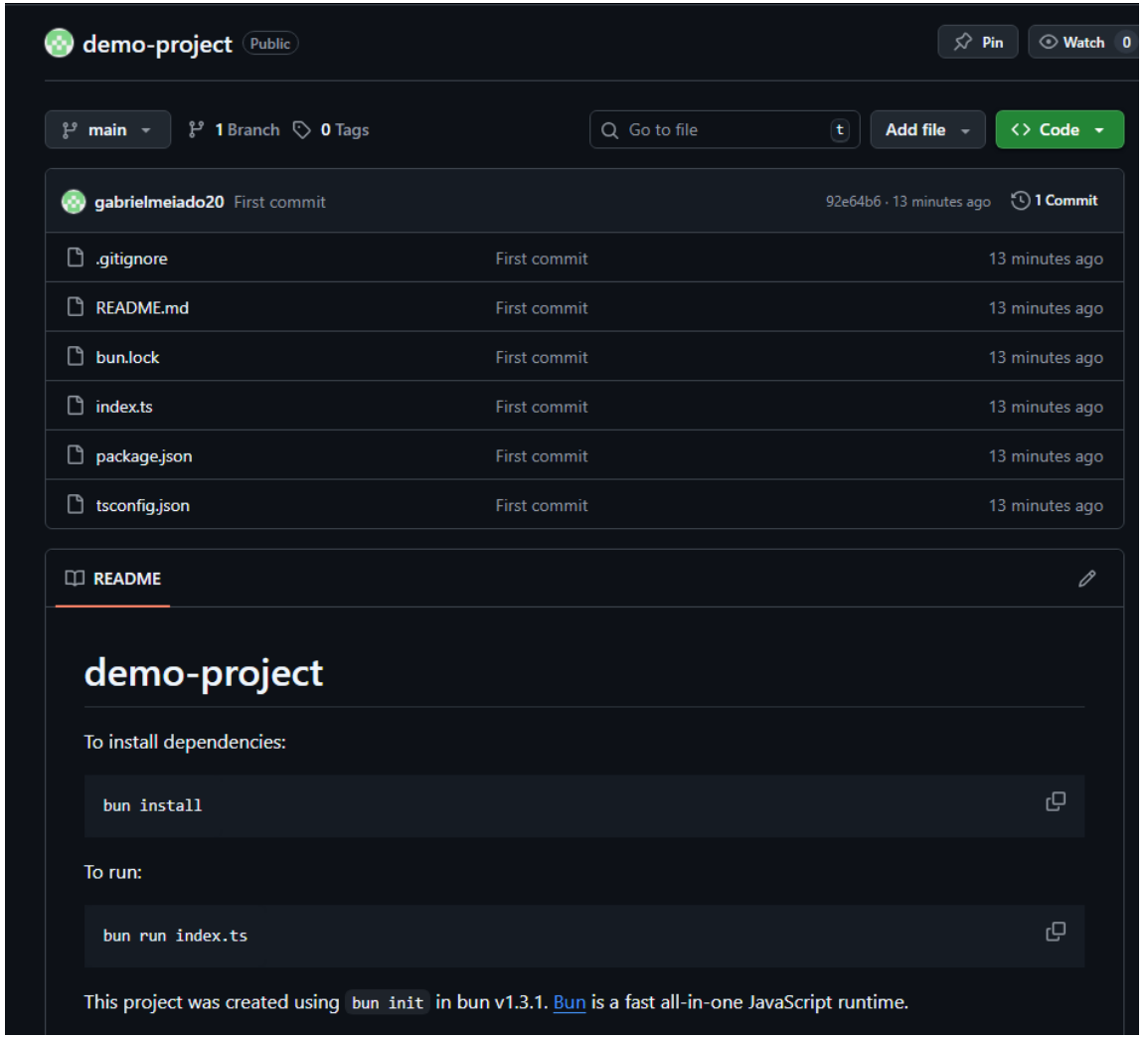
A primeira vez que fazemos usamos a flag ***–u*** para salvar nossa escolha indicando ao Git que a branch ***main*** local deve seguir a branch ***main*** remota em ***origin***

```
# Envia os commits da “main” local para a “origin”
# O –u (ou --set-upstream) conecta as duas branches
git push -u origin main

# Depois da primeira vez, podemos usar apenas:
git push
```

Repositórios Remotos.





Repositórios Remotos.



Até agora, nós "subimos" um projeto que criamos localmente. Mas, e se quisermos trabalhar num projeto que já existe?

O comando ***git clone*** faz exatamente isso. Ele "clona" (cria uma cópia local) de um repositório remoto.

```
# Copia o projeto para uma nova pasta na sua máquina
git clone https://github.com/usuario/repositorio-do-projeto.git

# Ele faz três coisas num só passo:
# 1. Cria uma nova pasta com o nome do projeto.
# 2. Baixa todo o histórico de commits e ficheiros.
# 3. Configura automaticamente o remote "origin“.
```

Repositórios Remotos.

```
MINGW64:/e/projects/new-projects

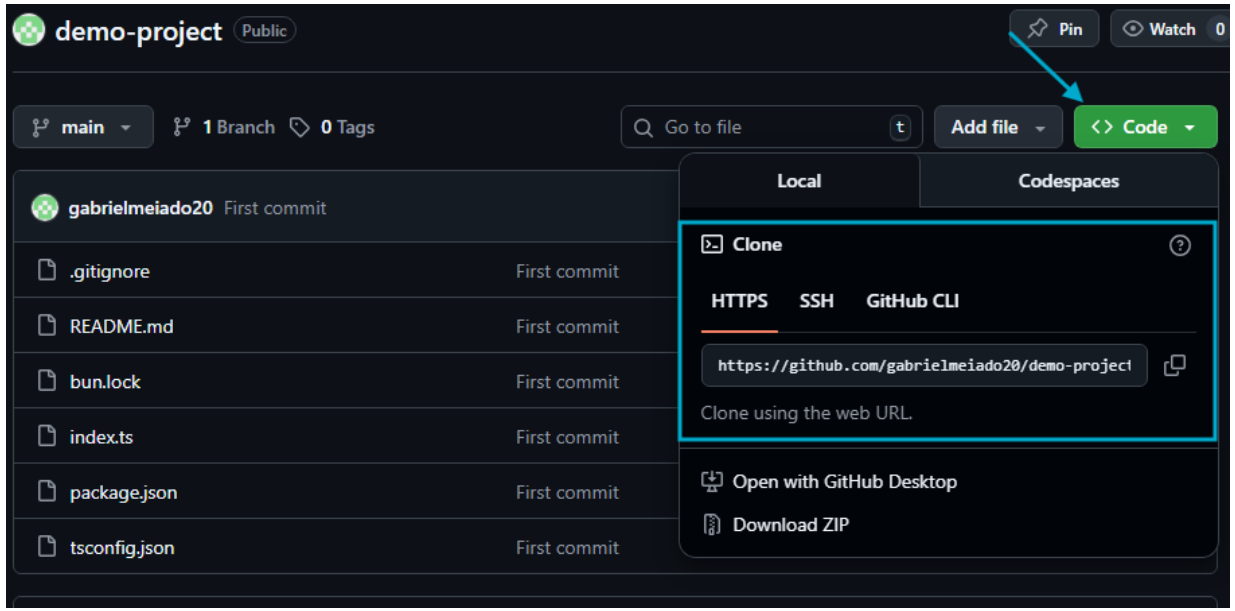
Gabriel@DESKTOP-04EJR4P MINGW64 /e/projects/new-projects
$ ls

Gabriel@DESKTOP-04EJR4P MINGW64 /e/projects/new-projects
$ git clone https://github.com/gabrielmeiado20/demo-project.git
Cloning into 'demo-project'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 8 (delta 0), reused 8 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (8/8), done.

Gabriel@DESKTOP-04EJR4P MINGW64 /e/projects/new-projects
$ ls
demo-project/

Gabriel@DESKTOP-04EJR4P MINGW64 /e/projects/new-projects
$ ls demo-project/
README.md  bun.lock  index.ts  package.json  tsconfig.json

Gabriel@DESKTOP-04EJR4P MINGW64 /e/projects/new-projects
$
```



Repositórios Remotos.

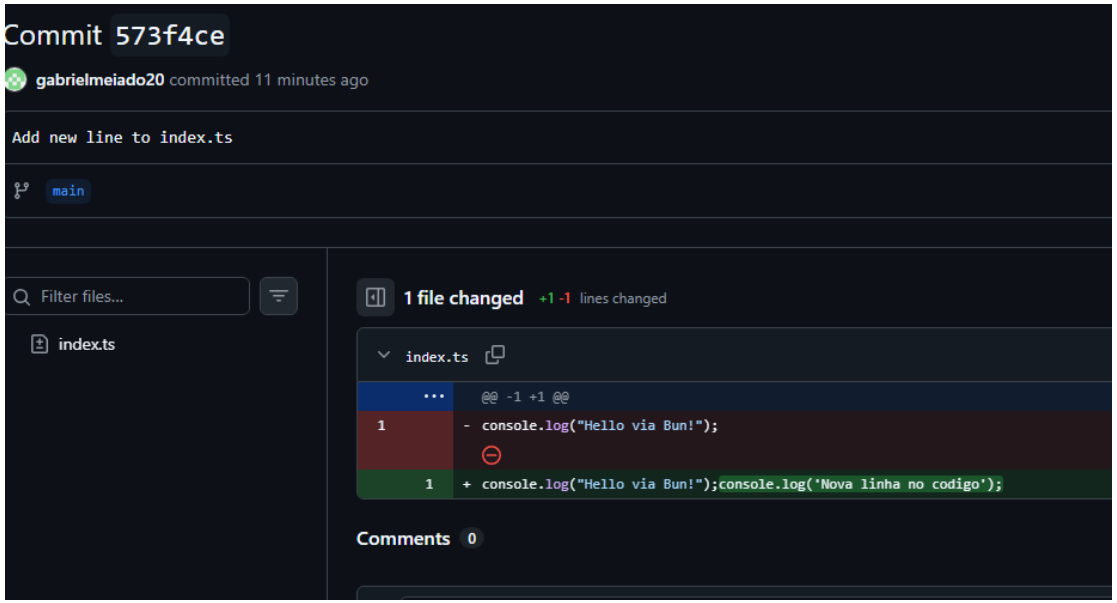
O ***git clone*** é feito apenas *uma vez*, no início.

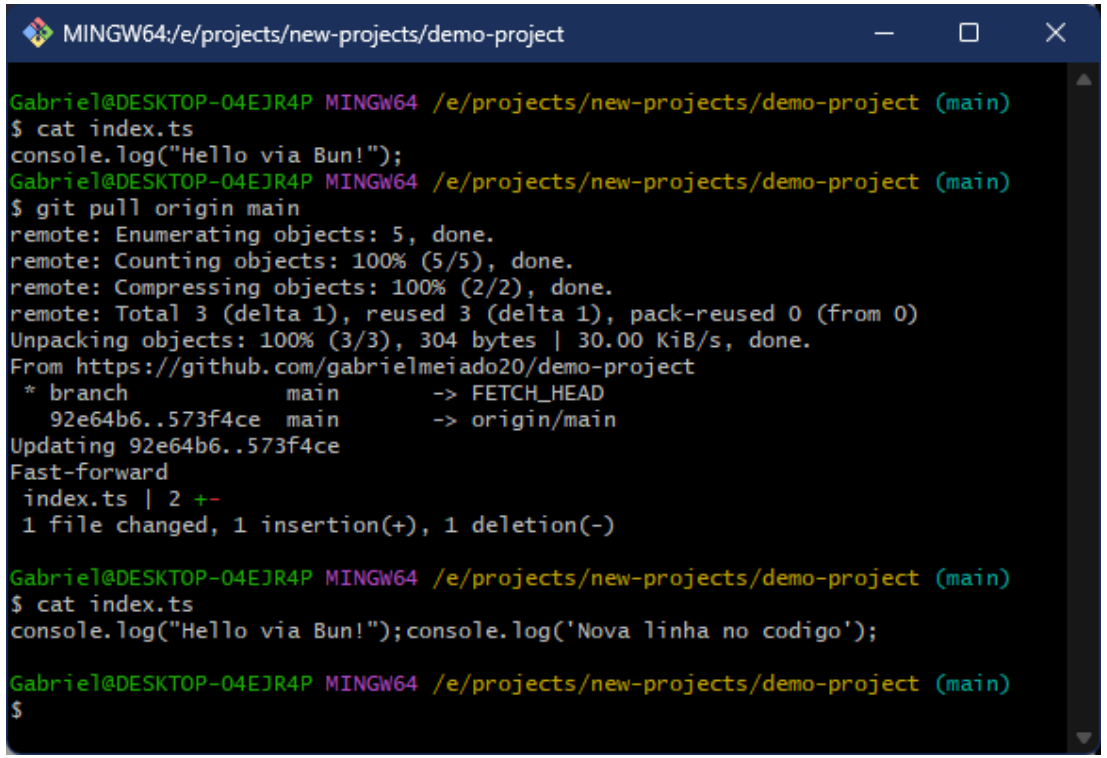
Depois disso, para trazer novas atualizações do repositório remoto (commits feitos por outros colegas, por exemplo), nós usamos o ***git pull***.

Este comando faz duas coisas: ele "busca" (***fetch***) as novidades do GitHub e depois "mescla" (***merge***) essas novidades na sua *branch* local. É a forma de garantir que você está sempre a trabalhar na versão mais recente do código.

```
# Busca e mescla as atualizações da branch
# “main” do remoto “origin” para a sua Branch
# local
git pull origin main
```

Repositórios Remotos.

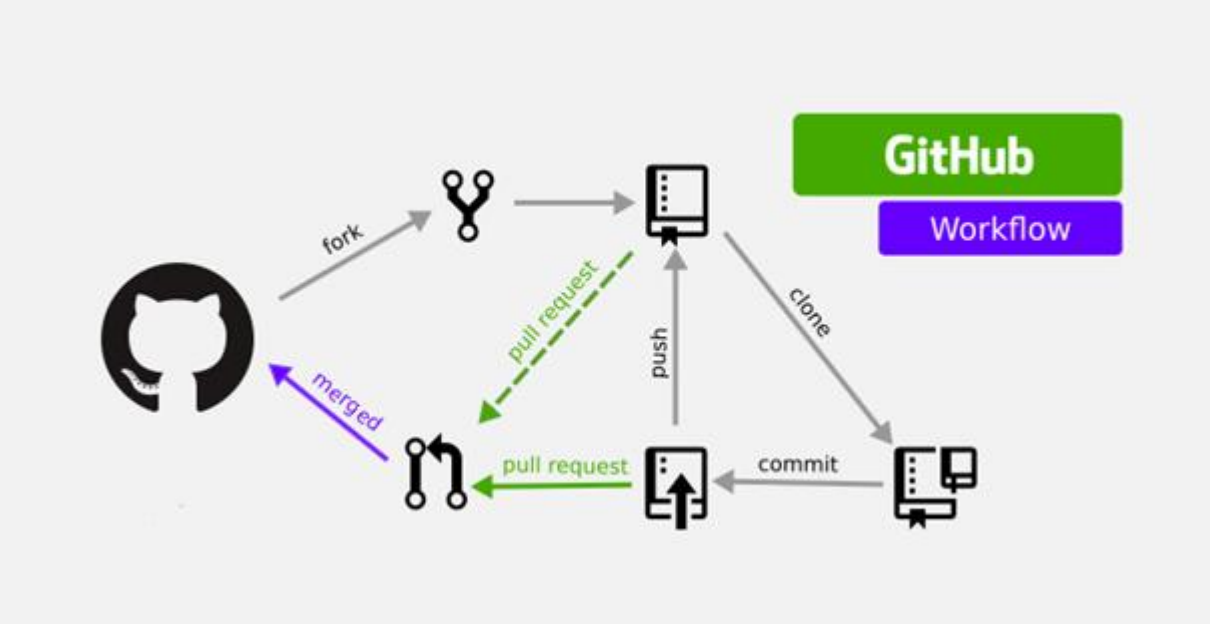




Fluxo de Colaboração.

Quando queremos contribuir para um projeto que **não é nosso** (um projeto open-source ou do seu local de trabalho), normalmente não temos as permissões para fazer um ***push*** direto

Nesse cenário nós seguimos um fluxo de trabalho diferente, envolvendo duas novas ideias: **Fork** e **Pull Request**



Fork e Pull Request.



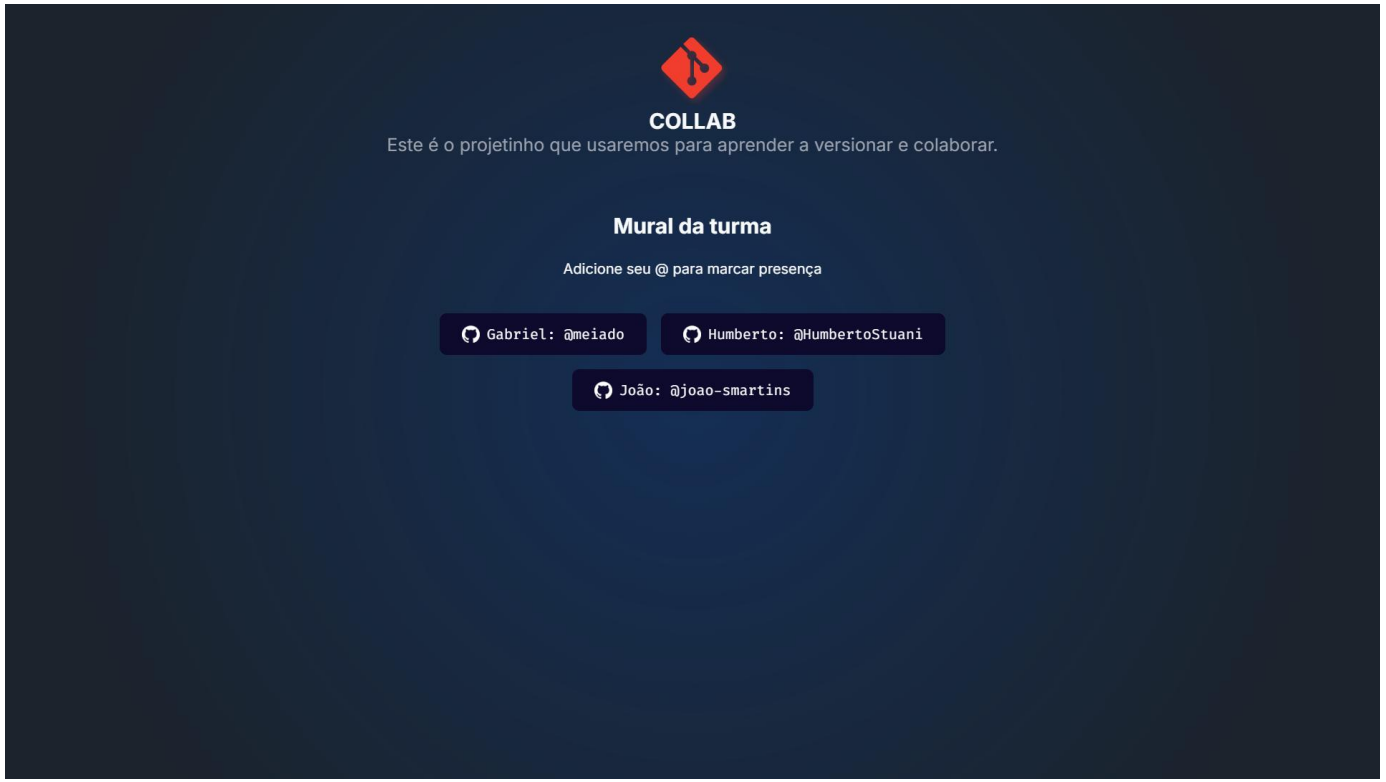
- **Fork** é uma **cópia pessoal** sua de um repositório de outra pessoa
- Fica na sua conta do GitHub
- Onde você tem permissão para salvar seu trabalho



- **Pull Request** é um pedido formal para o dono do projeto “puxar” suas alterações
- Você abre o pedido do **seu fork** para o **repositório original**
- Onde acontece o **Code Review** e a interação com outros colaboradores

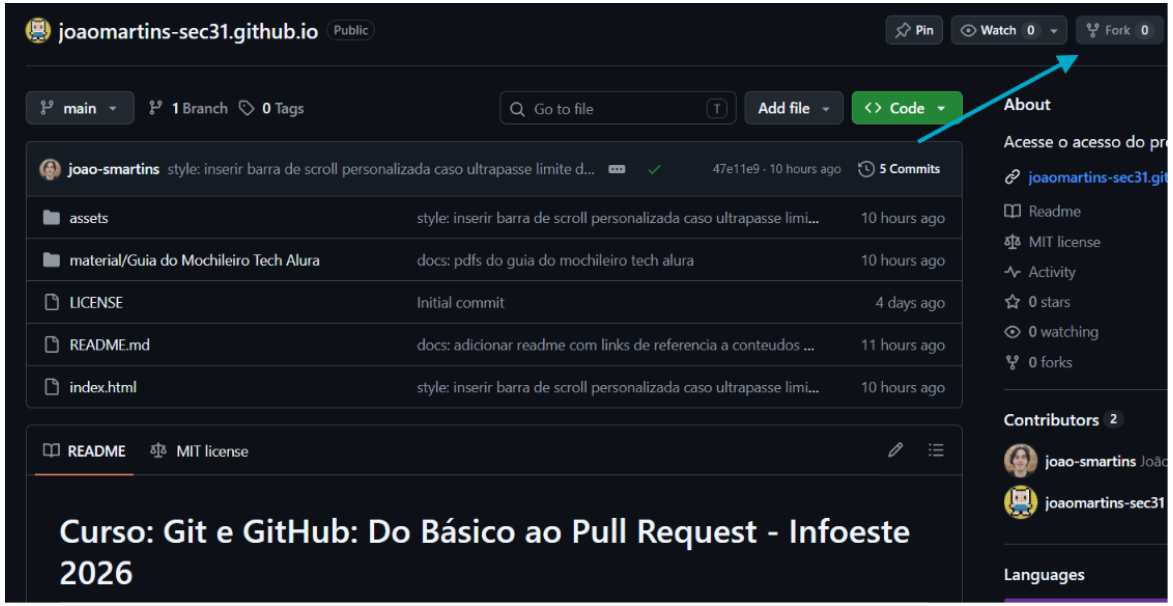
Colaborando.

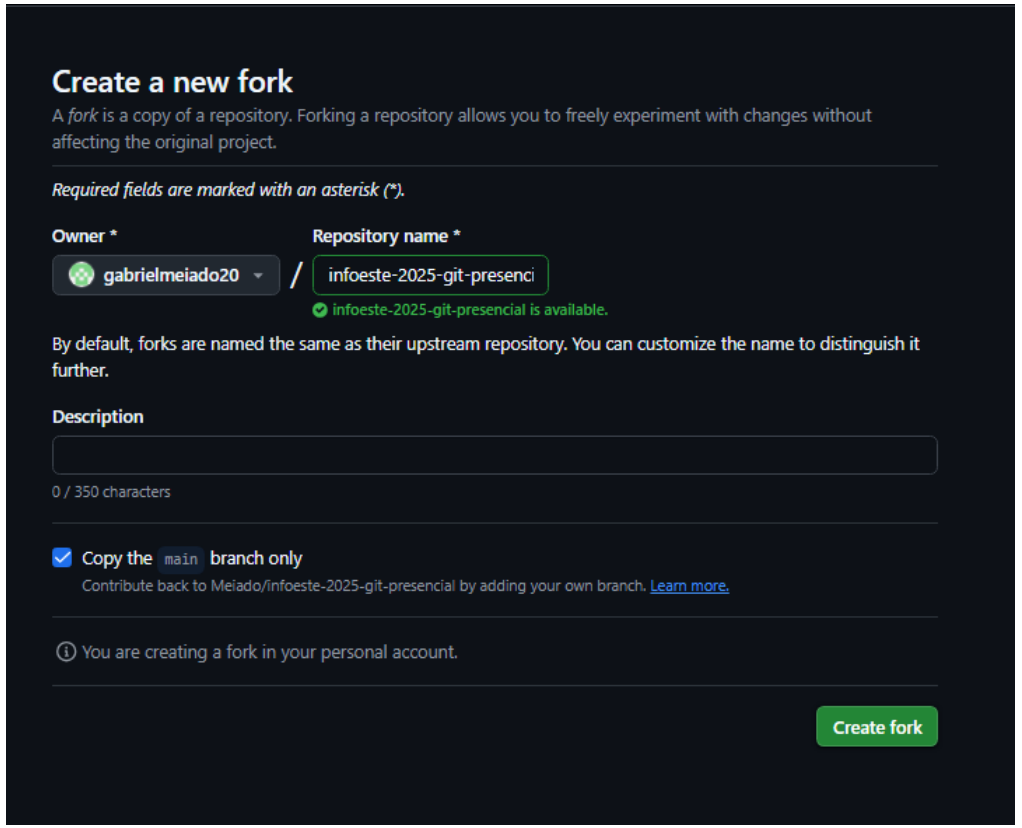
Fizemos um projeto para ser um mural da turma onde vocês podem colocar em prática tudo o que vimos até aqui.
O desafio agora é usar de tudo que vimos no decorrer do curso e fazer a contribuição de vocês com um projeto existente.



https://github.com/joaomartins-sec31/joaomartins-sec31.github.io

Colaborando.



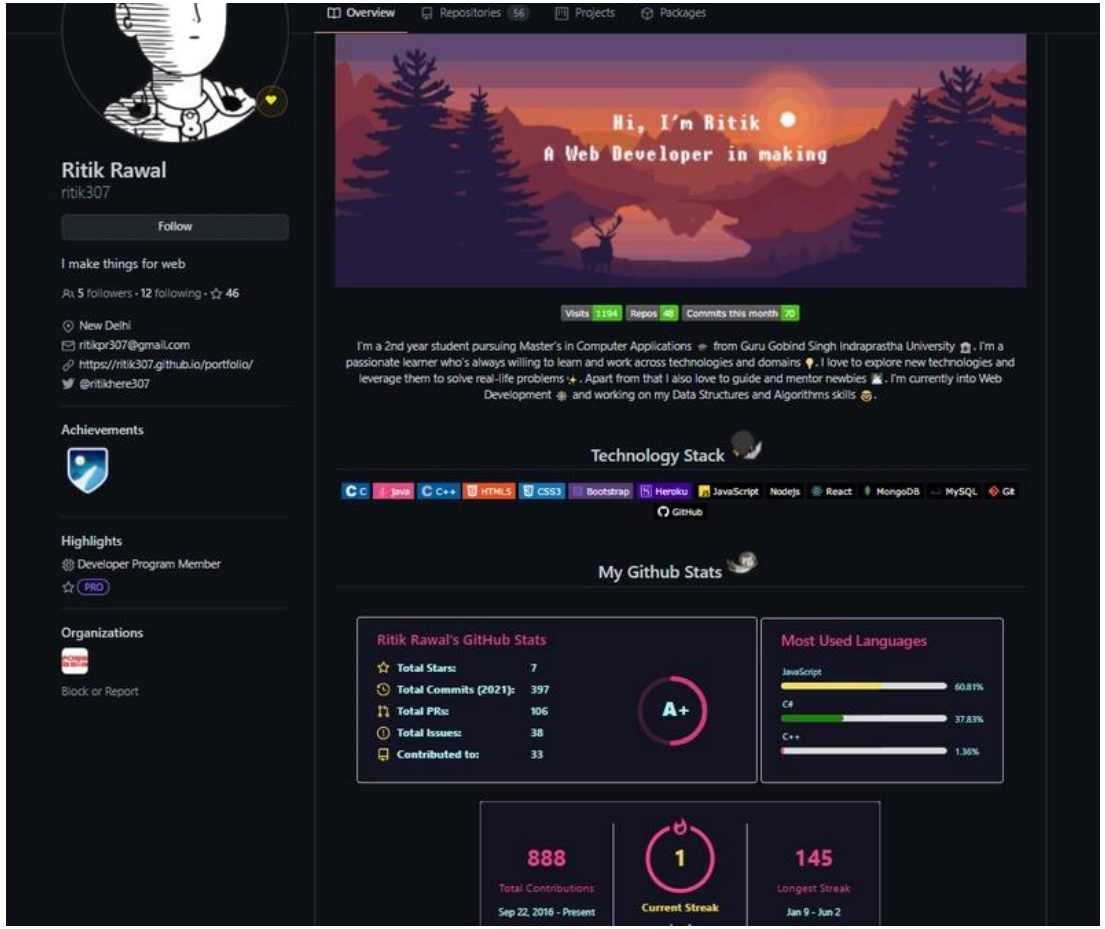


Construindo seu Portfólio.

Seu perfil no GitHub é seu Portfólio.

Recrutadores e empresas hoje olham o seu GitHub para ver o que você sabe fazer. Tudo o que aprendemos é a base para você mostrar seu trabalho ao mundo.

- Contribua para projetos open-source
- Crie seus próprios projetos
- Use o README.md
- Mantenha a “grama verde”



Perguntas?

"Consagre ao Senhor tudo o que você faz, e os seus planos serão bem-sucedidos." - Pv. 16:3

Obrigado pela presença!



João Martins
@joao-smartins
João Martins



Gabriel Meiado
@Meiado
Gabriel Meiado



Humberto Stuani
@HumbertoStuani
Humberto Stuani

INFOESTE 2025